

Master's Degree in Quantum Technologies

Academic year: 2024 - 2025

Title: **Symmetry-Optimized Classical Simulation
of Quantum Circuits**

Author: Boaz Ato Micah

Supervisors: Dr. Jara Juana Bermejo Vega, Dr. Giulio Camillo

Acknowledgments

I would like to express my deepest gratitude to Dr. Giulio Camillo for his constant support throughout this project. He not only helped me verify the results I obtained, but also played a crucial role in defining the scope of the work and provided continuous feedback on this report. Without his guidance, this project would not have been possible.

I would also like to give special appreciation to my supervisor, Dr. Jara Juana Bermejo Vega, for her invaluable supervision, guidance, and unwavering support at every stage of this work. I am truly grateful for her help and encouragement.

Finally, I would like to thank my family for their constant support and encouragement. Their patience and belief in me have been a source of strength, and I am deeply grateful to them.

Abstract

This thesis investigates the classical simulation of quantum circuits dominated by Clifford gates but supplemented with non-Clifford operations, using the *sum-over-Clifford* method. In this approach, non-Clifford gates are expressed as linear combinations of Clifford operators, allowing quantum circuits to be simulated as superpositions of stabilizer states. We develop and implement optimization techniques, including linear programming formulations, to obtain efficient decompositions of multi-CZ gates up to six qubits. The method exploits symmetries in non-Clifford gates: specifically, real diagonal non-Clifford gates can be decomposed exclusively using real diagonal Clifford group. This approach allows us to achieve the decomposition of gates with higher number of qubits. We also obtain the decomposition of other important gates such as $P(\theta)$, $\sqrt{\text{SWAP}}$, T , and $CP(\theta)$.

The results show that the stabilizer extension, $\xi(W)$, grows moderately with the number of qubits, confirming theoretical expectations and quantifying the cost of non-Clifford resources. Solver benchmarking reveals that HiGHS and GLPK yield sparse and efficient decompositions, while Clarabel produces denser solutions despite achieving the same optimal norms. Additionally, we demonstrate that global decompositions of entire circuits outperform the naive product of individual gate decompositions by exploiting circuit-level symmetries. Parallelization experiments with the HiGHS solver further establish the scalability of this method on High Performance Clusters.

These findings highlight the practical potential of the sum-over-Clifford framework as a bridge between stabilizer simulability and universal quantum computation. By quantifying the classical cost of non-Clifford resources, this work provides insight into the boundary of quantum advantage and offers concrete strategies for simulating near-term quantum circuits with improved efficiency.

Contents

1	One page summary in English	1
2	One page summary in Spanish	2
3	Introduction	3
4	Preliminary Methods	4
4.1	Universality	4
4.2	The Clifford Group	4
4.3	Generating the Clifford Group	5
4.4	Generating the Diagonal Clifford Group	5
4.5	Simulation in the Schrödinger Representation	6
4.6	Stabilizer-State Formalism	7
4.7	Gottesman-Knill Theorem	8
5	Methods	8
5.1	Classical simulation of Clifford dominating circuits	9
5.2	Sum Over Cliffords	10
5.3	Linear Programming	11
5.4	Multi-CZ gates	12
5.5	Solvers	13
5.6	Profiling	13
5.7	Hardware Specification	13
6	Result	14
6.1	Single and Two-Qubit Gates	14
6.2	Decomposition of multi-CZ gates	16
6.3	Multiplicity and Decomposition of Circuits	18
6.4	Parallelisation of the HIGHS solver	19
7	Discussion & Conclusion	20

1 One page summary in English

Quantum computing has emerged as one of the most promising areas of research in the past two decades, with the potential to surpass classical computing for certain tasks. However, the construction of large-scale fault-tolerant quantum computers remains a major challenge due to noise, decoherence, and the absence of scalable error correction schemes. A complementary line of research focuses on the *classical simulation* of quantum circuits, which provides valuable insights into the boundary between classically simulable systems and those that may demonstrate a genuine quantum advantage.

In this thesis, we investigate the *sum-over-Clifford* method as a practical framework for simulating Clifford-dominated circuits with a limited number of non-Clifford gates. Circuits composed only of Clifford gates can be simulated efficiently due to the Gottesman–Knill theorem, but universality requires the addition of non-Clifford operations. The sum-over-Clifford approach addresses this by decomposing non-Clifford gates into linear combinations of Clifford unitaries, allowing the entire circuit to be represented as a weighted sum of stabilizer states. The cost of such a decomposition is quantified by the *stabilizer extent* $\xi(W)$, which measures the “distance” of a unitary from the Clifford group.

We develop optimization-based methods, including linear (or more generally convex) programming formulations, to compute decompositions of important non-Clifford gates such as $P(\theta)$, $CP(\theta)$, $\sqrt{\text{SWAP}}$, and multi-controlled-Z (multi-CZ) gates up to six qubits. Benchmarking across solvers (HiGHS, GLPK, and Clarabel) reveals significant differences: HiGHS and GLPK consistently produce sparse and efficient decompositions, while Clarabel, though achieving the same optimal values, yields denser representations. Profiling experiments confirm that HiGHS benefits from parallelization, achieving faster runtimes with minimal memory overhead.

The results confirm theoretical predictions: the stabilizer extension grows moderately with the number of qubits, from $\xi(\text{CCZ}) = 1.778$ to $\xi(\text{CCCCCZ}) = 2.641$. Moreover, tensor-product decompositions here tested obey the multiplicativity property, while global decompositions of entire circuits often outperform the naive product of individual gates by exploiting circuit-level symmetries. This highlights the importance of structural optimization beyond gate-by-gate decomposition.

In conclusion, this work demonstrates that the sum-over-Clifford framework, combined with symmetry exploitation and efficient optimization solvers, provides a scalable method for simulating moderately sized non-Clifford circuits. These results not only clarify the role of non-Clifford resources in universal quantum computation but also offer practical strategies for simulating near-term quantum devices. Future extensions include generalization to non-diagonal gates, integration with tensor network techniques, and deployment on high-performance computing clusters. In this way, the thesis contributes to a deeper understanding of the boundary between classical and quantum computational power, while providing tools to push this frontier further.

2 One page summary in Spanish

La computación cuántica se ha convertido en una de las áreas de investigación más prometedoras en las últimas dos décadas, con el potencial de superar a la computación clásica en ciertas tareas. Sin embargo, la construcción de ordenadores cuánticos a gran escala con tolerancia a fallos sigue siendo un gran desafío debido al ruido, la decoherencia y la ausencia de esquemas escalables de corrección de errores. Una línea de investigación complementaria se centra en la simulación clásica de circuitos cuánticos, que proporciona información valiosa sobre la frontera entre los sistemas simulables clásicamente y aquellos que pueden demostrar una auténtica ventaja cuántica.

En esta tesis, investigamos el método de *sum-over-Cliffords* como framework práctico para simular circuitos Clifford-dominados con un número limitado de puertas no Clifford. Los circuitos compuestos únicamente por puertas Clifford pueden simularse eficientemente gracias al teorema de Gottesman-Knill, pero la universalidad requiere la adición de operaciones no Clifford. El enfoque de sum-over-Clifford aborda este problema descomponiendo las puertas no Clifford en combinaciones lineales de unitarias de Clifford, lo que permite representar el circuito completo como una suma ponderada de estados estabilizadores. El coste de dicha descomposición se cuantifica mediante el *stabilizer extent* $\xi(W)$, que mide la distancia entre una unitaria y el grupo de Clifford.

Desarrollamos métodos basados en optimización, incluyendo formulaciones de programación lineal (o, más generalmente, convexa), para calcular descomposiciones de puertas no Clifford importantes, como $P(\theta)$, $CP(\theta)$, $\sqrt{\text{SWAP}}$ y puertas multi-controladas-Z (multi-CZ) de hasta seis cúbits. La evaluación comparativa entre solvers (HiGHS, GLPK y Clarabel) revela diferencias significativas: HiGHS y GLPK producen consistentemente descomposiciones dispersas y eficientes, mientras que Clarabel, si bien alcanza los mismos valores óptimos, produce representaciones más densas. Los experimentos de perfilado confirman que HiGHS se beneficia de la paralelización, logrando tiempos de ejecución más rápidos con una mínima sobrecarga de memoria.

Los resultados confirman las predicciones teóricas: el stabilizer extent crece moderadamente con el número de cúbits, de $\xi(\text{CCZ}) = 1,778$ a $\xi(\text{CCCCCZ}) = 2,641$. Además, las descomposiciones producto-tensorial probadas aquí obedecen a la propiedad de multiplicidad, mientras que las descomposiciones globales de circuitos completos suelen superar el producto ingenuo de puertas individuales al explotar simetrías a nivel de circuito. Esto resalta la importancia de la optimización estructural más allá de la descomposición puerta por puerta.

En conclusión, este trabajo demuestra que el framework de sum-over-Cliffords, combinado con la explotación de simetría y solvers de optimización eficientes, proporciona un método escalable para simular circuitos no Clifford de tamaño moderado. Estos resultados no solo aclaran el papel de los recursos no Clifford en la computación cuántica universal, sino que también ofrecen estrategias prácticas para simular dispositivos cuánticos a corto plazo. Las futuras extensiones incluyen la generalización a puertas no diagonales, la integración con técnicas de redes tensoriales y la implementación en clústeres de computación de alto rendimiento. De esta manera, la tesis contribuye a una comprensión más profunda de la frontera entre la potencia computacional clásica y cuántica, a la vez que proporciona herramientas para ampliar esta frontera.

3 Introduction

Quantum computing is one of the most thriving fields in physics over the last 20 years [1]. Since its conception to the present day, major milestones have been achieved [2], yet there is still a long road ahead to build a fault tolerant quantum computer that shows a clear computational advantage over classical computers. One of the main reasons is that quantum systems are highly sensitive to noise, which inevitably causes errors in quantum algorithms. To realize large-scale systems, we must achieve error rates much lower than have been demonstrated thus far in a scalable platform [3]. In addition to noise, we require a general formalism for error correction applicable to arbitrary quantum circuits [4].

Due to these difficulties and the need to realize the potential of quantum computers, researchers have been making significant efforts on how to efficiently simulate quantum circuits on classical computers. This will allow scientists to better design and test quantum circuits. Some current research on this topic has focused on developing efficient algorithms to simulate stabilizer circuits. These circuits are unique in that they can be simulated in polynomial time [5]. However, stabilizer circuits alone cannot perform universal quantum computation.

Another reason to study the simulation of quantum circuits is the BQP (class of functions solvable by a quantum circuit in polynomial time with an error probability bounded by $\frac{1}{3}$) vs the BPP (class of problems solved by randomized algorithms with an error probability bounded by $\frac{1}{3}$) problem. While we know $BPP \subseteq BQP$ because we can simulate any problem in BPP with a quantum circuit in polynomial time, it is still unknown whether BQP is a strict superset of BPP [6]. Simulating quantum circuits on classical computers can then be seen as an effort to contain the power of quantum circuits, as successful and universal simulation of quantum circuits would imply that BQP cannot be seen as a strict superset of BPP.

Regardless, it is widely believed that universal quantum computers cannot be efficiently simulated by probabilistic algorithms. This belief is partly supported by the fact that state-of-the-art classical simulators employing modern supercomputers are still limited to a few dozens of qubits [7].

At the same time, certain quantum information processing tasks do not require computational universality. For example, quantum error correction based on stabilizer codes and Pauli noise models only requires quantum circuits composed of Clifford gates and Pauli measurements—which can be easily simulated classically using the Gottesman-Knill theorem [8]. Furthermore, Clifford circuits become universal for quantum computation when supplemented with abundant resources beyond stabilizer operations, such as non-Clifford gates or a supply of magic states [9]. This raises the possibility of simulating quantum circuits with a large number of qubits and few non-Clifford gates. Aaronson and Gottesman [10] were the first to propose a classical simulation method covering this situation, with a runtime that scales polynomially with the number of qubits and Clifford gates, but exponentially with the number of non-Clifford gates. This early work is an intriguing proof of principle but with a very large exponent, limiting potential applications.

In the current state of research, there is still no clear consensus on what makes a quantum computer superior to a classical one. We know of algorithms such as Shor’s [11], which, if implemented, would find the prime factors of large integers in polynomial time. Also, it is well known that, being an inherently quantum machine, there is a natural advantage when it comes to simulating other quantum systems such as molecules, atoms, or open quantum systems [12]. However, the implementation problem remains unsolved.

An important notion here is that if a quantum circuit can be simulated on a classical computer in polynomial time, then there is no quantum advantage. By in “polynomial time,” we mean that the time and resources required for the simulation scale polynomially with the number of inputs or the number of qubits required to state the problem.

The premise of this Master’s thesis is: quantum circuits composed only of Clifford gates do not constitute a model of universal quantum computation. Clifford gates and non-Clifford gates are needed for universality. This project focuses on achieving a practical decomposition of non-Clifford gates, specifically multi-CZ gates, into a linear combination of Clifford gates. This approach, known as the sum-over-Clifford approach, can be used to simulate quantum circuits. This thesis explores symmetries in circuits in order to decompose more

complicated non-Clifford gates, motivated by work being done in the research group [13]. The key idea is that real-diagonal non-Clifford gates can be decomposed into a linear combination of real-diagonal Clifford gates. This approach allows the decomposition of gates acting on a larger number of qubits, surpassing previous limitations.

The structure of this project is as follows: Section 4 presents the preliminary methods required for understanding the sum-over-Cliffords approach. This includes an overview of universality, the Clifford Group, simulation in the Schrödinger representation, the Stabilizer-State Formalism, and the Gottesman-Knill Theorem. Section 5 outlines the methods employed in the project, beginning with the classical simulation of Clifford-dominating circuits. The sum-over-Cliffords method is then introduced, followed by a discussion of linear programming and convex optimization techniques. It also includes a discussion of the solvers used to solve the optimization problem and the profiling method used to track the computational resources used by the solvers. Section 6 presents the results obtained, including simulations of controlled-Z gates up to six qubits using three different solvers. The results also include the decomposition of notable one and two qubit gates. Finally, Section 7 provides the discussion and conclusion.

4 Preliminary Methods

4.1 Universality

A universal quantum computer is capable of converting an input state into an arbitrarily chosen output state. This means that the quantum computer must have access to a set of gates that, when combined, can realize any desired operation. Since quantum gates correspond to unitary operators, and these form a continuous group, there are infinitely many possible gates in principle. However, universality does not require an infinite set of gates. In practice, one instead selects a finite gate set that can approximate any unitary operation to arbitrary precision. Such a set is called universal. The approximation is achieved by forming products of gates, so that the group generated by the universal set is dense in the full unitary group.

4.2 The Clifford Group

Given the Pauli matrices:

$$\sigma_0 = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}, \quad \sigma_1 = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, \quad \sigma_2 = \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix}, \quad \sigma_3 = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}. \quad (1)$$

The Pauli group for n qubits, P_n is defined as:

$$P_n = \{e^{i\theta/2}\sigma_{j_1} \otimes \dots \otimes \sigma_{j_n} | \theta = 0, 1, 2, 3, j_k = 0, 1, 2, 3\}. \quad (2)$$

The elements of the Pauli group are tensor products of the Pauli matrices multiplied by a global phase.

The Clifford group for n qubits, in its matrix representation, is defined as:

$$C_n = \{V \in U(2^n) | VP_nV^\dagger = P_n\}, \quad (3)$$

where $U(2^n)$ is the set of unitary matrices of dimension 2^n . Any element of the Clifford group will be referred to as a Clifford gate. These set of gates normalizes the Pauli group. That is to say, the Clifford group maps elements of P_n to other elements of P_n . Clifford gates can be efficiently simulated classically, and the formalism of quantum error correction has been developed around them. However, they do not form a universal set. The addition of non-Clifford gates such as the CCZ gate and the T gate, makes them universal.

4.3 Generating the Clifford Group

It turns out that the entire Clifford group can be generated from a small set of basic gates called the generators. The generators of the Clifford group, in the computational basis $\{|0\rangle, |1\rangle\}$ for one and two qubits are:

$$H = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}, \quad S = \begin{pmatrix} 1 & 0 \\ 0 & i \end{pmatrix}, \quad CZ = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & -1 \end{pmatrix}, \quad \omega = e^{i\pi/4} \quad . \quad (4)$$

Where H is the Hadamard gate, S is the phase gate, CZ is the Controlled-Z gate and w is a global phase.

To produce the full matrix group for a particular number of qubits n , one can simply consider all products of these generators (in all possible orders, with all relevant exponents) and discard any duplicates. An important optimization is to determine the exponent at which each generator returns to the identity, thereby reducing unnecessary calculations. Using this approach, one can *a priori* estimate the maximum number of combinations needed to list all group elements.

The Clifford group on n qubits is finite but grows *super-exponentially* in n . Indeed, for $n > 3$, storing its entire set of elements on a standard computer becomes infeasible. Furthermore, each element of the group is a $2^n \times 2^n$ matrix, implying 2^{2n} complex entries (or 2^{2n+1} floating-point numbers if we store real and imaginary parts separately). The equation below quantifies the growth:

$$|C_n| = \prod_{j=1}^n 2^{\frac{4j-1}{4}} = 2^{n^2+2n} \prod_{j=1}^n (4j-1). \quad (4.5)$$

This equation counts the *phase-free* Clifford group on n qubits. Table 1 shows the cardinality of the phase-free group for some small n . Notice that even at $n = 3$, the group size is on the order of 10^8 elements, each a $2^3 \times 2^3 = 8 \times 8$ matrix of complex numbers. This already demands tens of gigabytes of storage, and the memory requirement escalates dramatically for larger n .

n	1	2	3	4	5
$ C_n $	24	11,520	92,897,280	1.2129×10^{13}	2.5411×10^{19}

Table 1. Cardinality of the phase-free Clifford group for small numbers of qubits.

For $n = 3$, this is borderline practical; beyond that, it becomes nearly impossible to do exhaustive computations.

For the case $n \leq 2$, one can still generate and store this group fully. In this project, I used the library *QuaEC* (*Quantum Error Correction Analysis*) in Python to obtain the full Clifford group for one and two qubits[14]. These Clifford groups were then used to perform the decomposition of some important non-Clifford gates like the T and CZ gates.

4.4 Generating the Diagonal Clifford Group

In quantum computing, many gates of interest are diagonal in the computational basis. Thus, it is often both practical and interesting to focus on decomposing diagonal gates into the *Diagonal Clifford Group*, which is substantially smaller than the full group. Furthermore, dealing with diagonal matrices greatly reduces both memory usage and computational time [13].

The generators of the Diagonal Clifford Group are the following two diagonal operators:

$$\langle S_i, CZ(i, j), \omega = e^{i\pi/4} \rangle. \quad (5)$$

The subscripts refer to the qubit(s) on which the gate acts on. In this project, the phases are omitted and introduced later if necessary.

Note that the Phase gate, P , has order 4, since $P^4 = I$. Similarly, the controlled-Z gates, $CZ(i, j)$, each have order 2. As these diagonal matrices all commute, the order of multiplication does not matter, and generating the diagonal Clifford group is straightforward.

Another advantage of the diagonal structure is that matrix multiplication is reduced to a simple element-wise product of the diagonals, reducing the complexity from $O(n^3)$ (for general matrix multiplication) to $O(n)$.

Although we have not found a published formula for the exact cardinality of the diagonal Clifford group, they can be easily calculated by counting possible exponents. Denoting by $|C_n^D|$ the cardinality for the phase-free *Diagonal Clifford Group* on n qubits, one observes that:

- Each qubit introduces a Phase gate P_i (or S_i) of order 4.
- Each unordered pair of qubits (i, j) introduces a $CZ(i, j)$ gate of order 2.

Hence, there are n possible single-qubit diagonals (each with 4 possible exponents) and $\binom{n}{2}$ possible two-qubit diagonals (each with 2 possible exponents). Thus, the group's size (or at least an upper bound on its size) is

$$|C_n^D| \leq 4^n \times 2^{\binom{n}{2}} = 2^{2n + \binom{n}{2}}. \quad (6)$$

The table below shows these bounds for n up to 6:

n	1	2	3	4	5	6
$ C_n^D $ (bound)	4	32	512	16,384	1,048,576	134,217,728

Table 2. Upper bound on the number of elements in the phase-free Diagonal Clifford Group.

Thus, we see that for $n = 5$, one reaches over a million elements. Although still large, it is drastically smaller than the full Clifford group for $n = 5$. Indeed, generating, storing, or operating on these diagonal matrices is far more feasible.

4.5 Simulation in the Schrödinger Representation

Before introducing the sum-over-Clifford method, let us first discuss the obvious approach to simulating quantum circuits. Any quantum circuit can be reduced to the product of its gates, which act on the input state vector. If gates $U_1, U_2, \dots, U_{n-1}, U_n$ are applied in chronological order, then when applying them to the input state, one must multiply from right to left:

$$U_n U_{n-1} \cdots U_2 U_1 |\psi\rangle = U_t |\psi\rangle. \quad (7)$$

In general—and this is necessary for actual computation on a computer—it is useful to represent all gates in the full dimension of the system. That is, if we have the gate T_2 , we represent it as $I_1 \otimes T_2 \otimes I_3 \otimes \cdots \otimes I_n$.

The most direct method to simulate a quantum circuit is as follows: simply apply all matrices, in coordinate form, to the state vector. Performing this product yields the final state, from which one can project onto any desired component or compute the expectation value of any observable serving as a measurement.

There is a clear limitation of this approach and that is storing all the matrices that act on the state and on the state vector itself. For just 50 qubits, we would need a matrix of size $m \times m = (2^{50})^2 \approx 1.126 \times 10^{30}$ complex numbers, which is unmanageable for a conventional computer.

This highlights the inherent problem of dealing with quantum systems on classical computers: the memory requirements explode exponentially as the number of qubits increases. This is a clear indication of the potential advantage of using quantum systems, which store much more information in fewer elements than their classical counterparts.

In practice, numerous alternatives exist to this kind of brute-force simulation, which is essentially nothing more than a direct application of the postulates of quantum mechanics. In the following sections, we will see how, over the past few decades, much more sophisticated algorithms have been devised by exploiting the symmetries and properties of subgroups of $U(2^n)$ and by restricting the set of gates and input states used in the circuits.

4.6 Stabilizer-State Formalism

Before continuing with other simulation methods, we need to define the stabilizer-state formalism and its close relationship to error correction, which is the main challenge in the physical implementation of quantum computers.

Following the approach of Nielsen and Chuang[15], we revisit an example from their book. Consider one of the EPR states, well known from the quantum teleportation algorithm:

$$|\psi\rangle = \frac{|00\rangle + |11\rangle}{\sqrt{2}}. \quad (8)$$

It can be verified that the operations $X_1 X_2 |\psi\rangle$ and $Z_1 Z_2 |\psi\rangle$ each yields $|\psi\rangle$. In other words, the action of these operators on the state leaves it unchanged; they *stabilize* it. It can be shown (though that is not the focus here) that $|\psi\rangle$ is the only state stabilized by those two operators.

Stabilizer formalism does not emphasize the direct representation of the state itself; rather, it describes the ket vectors in terms of the operators that stabilize them. Notice from the example that to represent the two-qubit state in the usual coordinate form, we need four complex numbers, whereas only two stabilizing operators suffice to define it uniquely.

Leaving this introduction behind, we define what is meant by a *stabilizer*: Suppose S is a subgroup of P_n , the Pauli group on n qubits (as defined in Equation 2). Let V_S be the vector subspace of n -qubit states that remain invariant under the action of every operator in S . Essentially, V_S is the space of eigenvectors with eigenvalue 1 for all operators in S . One says that S *stabilizes* V_S .

A group is uniquely specified by its set of generators. This is highly advantageous because if a state is stabilized by certain operators individually, it is also stabilized by any product of them. Thus, to check whether a vector is stabilized by S , one only needs to know the generators of S , not every element.

Focusing on our main topic, we examine the relationship between unitary operators (quantum gates) and the stabilizer formalism. Let U be a unitary operator, and let V_S be the vector space stabilized by a subgroup S of the Pauli group. Then for any element $g \in S$:

$$U |\psi\rangle = Ug |\psi\rangle = UgU^\dagger U |\psi\rangle. \quad (9)$$

Hence, the state $U |\psi\rangle$ is stabilized by $UgU^\dagger$. Revisiting the definition of the Clifford group in Equation 3—what initially looked somewhat opaque now becomes clearer. If $U \in C_n$, then $UgU^\dagger \in P_n$. Therefore, a

state stabilized by some subgroup of the Pauli group remains a Pauli-stabilized state after the application of Clifford gates.

At first glance this might appear rather trivial, but it is in fact a critically important result. If a state is stabilized by the generators $\langle Z_1, Z_2, \dots, Z_n \rangle$, then one can easily verify that this state is $|0\rangle^{\otimes n}$. By applying the Hadamard gate to each of the qubits, the resulting state is stabilized by the action $HZH^\dagger$ on each qubit, i.e. by $\langle X_1, X_2, \dots, X_n \rangle$. Hence, we only need to track n elements rather than 2^n elements, as would be required in a vector-space (coordinate) representation.

Simulating a circuit that employs stabilizer states and only gates from the Clifford group is efficient at first glance. The state is labeled by the generators of the subgroup that stabilizes it. Then, to track the circuit's evolution, one applies the gates of the circuit to the generators corresponding to the qubits on which they act, keeping track of only n elements: the stabilizer operators. Furthermore, the action of the Clifford group generators on the Pauli group is well understood and can be implemented readily.

4.7 Gottesman-Knill Theorem

Now, we introduce the Gottesman-Knill Theorem [8]. Suppose a quantum computation only involves the following:

- (i) state preparation in the computational basis
- (ii) H gates, S gates, CNOT gates, Pauli gates
- (iii) measurements of Pauli observables and
- (iv) classical control conditioned on the outcome of such measurements

Theorem (Gottesman-Knill). *Such a quantum system is efficiently simulable by a classical computer.*

A key insight of this theorem is the tracking of the evolution of Pauli operators that stabilize the qubits in use. It is shown that the complexity of simulating such a circuit on a classical computer is $O(n^2m)$, where n is the number of qubits and m is the number of gates in the circuit [10]. Thus, any algorithm that employs the elements stated in the theorem does not offer a quantum advantage over a classical algorithm, since it can be simulated classically in an efficient manner. It is worth emphasizing that, by using only CNOT and Hadamard gates on a computational-basis state, one can generate a highly entangled state. Thus, this theorem shows that entanglement alone is not sufficient to guarantee a speedup over classical computers. In the search for additional properties that could confer an advantage on quantum circuits, the conditions of the theorem are deliberately violated, and researchers investigate whether there exists some measurable quantum resource that makes certain circuits non-classically simulable. If they were simulable, then the Gottesman-Knill theorem could be extended to those cases as well. What might work?

1. Investigate what happens if the input states are neither stabilizer states nor states in the computational basis.
2. Use a gate set that confers universality; i.e., Clifford + T, Clifford + CCZ, etc.

By making these changes, one can explore whether such circuits remain classically simulable. If they are not, then there may be a quantifiable ingredient necessary for achieving a quantum advantage. If they are, the Gottesman-Knill theorem can be extended further.

5 Methods

In this section, I explain the sum-over-Clifford method that was used to obtain the decomposition of the multi-CZ gates.

5.1 Classical simulation of Clifford dominating circuits

One approach to probing the limits of classical simulation of quantum algorithms is the *Sum-Over-Cliffords* method [16]. In these circuits, we employ Clifford gates together with a small number of non-Clifford gates, which together can provide a certain level of universality.

A circuit can be structured as follows:

$$U = D_m V_m D_{m-1} V_{m-1} \cdots D_1 V_1 D_0, \quad (10)$$

where it acts on the input state $|0\rangle^{\otimes n}$. The operators D_j are Clifford circuits, and the operators V_j are non-Clifford gates. The introduction of the non-Clifford gates makes the circuit not fulfill the requirements of the Gottesman-Knill Theorem. As such, we do not have the guarantee that the quantum circuit can be simulated efficiently by a classical computer.

We then write U in a sum-over-Clifford decomposition as follows:

$$U = \sum_j c_j K_j, \quad (11)$$

where each K_j is a Clifford unitary, and c_j are complex coefficients. The action of U on the input state is then

$$U |0^n\rangle = \sum_j c_j K_j |0^n\rangle. \quad (12)$$

With error δ , one can approximate the final state $U |0^n\rangle$ with a superposition of stabilizer states $|\psi\rangle$ consisting of

$$k \approx \delta^{-2} \|c\|_1^2 \quad (13)$$

terms, where $\|c\|_1$ is the 1-norm of the coefficients c_j :

$$\|c\|_1^2 = \sum_i |c_i|. \quad (14)$$

In short:

$$\|U |0^n\rangle - |\psi\rangle\| \leq \delta, \quad |\psi\rangle = \sum_{j=1}^k c_j K_j |0^n\rangle, \quad (15)$$

Essentially, the original unitary circuit is decomposed into a superposition of Clifford circuits so that the final state can be approximated by a stabilizer state.

As can be seen, the number of terms in the decomposition grows with $\|c\|_1^2$, meaning the algorithmic complexity grows with that norm. Minimizing this norm is therefore crucial to making the method tractable.

As noted, controlling the $\|c\|_1$ norm is crucial for minimizing the computational resources required for simulation. Moreover, this 1-norm again becomes significant when calculating expected values or probability distributions, because a larger norm implies a longer superposition of final stabilizers. Besides, to know the norm, we need the decomposition. So, being able to compute it gives us, as a byproduct, the decomposition itself, which can then be fed into the simulator.

One might attempt to compute the decomposition of the entire circuit U (in terms of Clifford unitaries) via a brute-force approach, but that would be highly inefficient. A common practical strategy is simpler: one decomposes each non-Clifford gate V_j individually into a sum of Clifford operators and incorporates these decompositions into the circuit to obtain a valid, though not necessarily optimal, global decomposition. Indeed, a more optimal solution might exist if the entire circuit were taken into account, but handling the entire circuit at once is often computationally prohibitive or outright infeasible. In this project, we explore both approaches: decomposing entire circuits and decomposing individual gates, and then compare the results.

Hence, if a given non-Clifford gate V_p can be decomposed as

$$V_p = \sum_j c_j^{(p)} K_j^{(p)}, \quad (16)$$

then by inserting these decompositions into Eq.10, one obtains

$$U = \sum_{j_1, \dots, j_m} \left(\prod_{p=1}^m c_{j_p}^{(p)} \right) D_m K_{j_m}^{(m)} D_{m-1} \cdots D_1 K_{j_1}^{(1)} D_0. \quad (17)$$

It follows that the 1-norm of the global coefficients factorizes:

$$\|c\|_1^2 = \prod_{p=1}^m \|c^{(p)}\|_1^2. \quad (18)$$

Since reducing $\|c\|_1$ is a priority in limiting the computational cost, we define:

Definition 1 (Stabilizer Extension). *For a unitary operator W , the stabilizer extension $\xi(W)$ is the minimum over $\|c\|_1^2$ among all decompositions of the form*

$$W = \sum_j c_j K_j, \quad K_j \text{ are Clifford unitaries.}$$

Note that to apply the Sum-Over-Cliffords method, one does not actually need the *optimal* decomposition of a unitary. Any valid decomposition will suffice. However, since $\|c\|_1$ is multiplicative [Eq.18], not using $\xi(W)$ (the minimal-norm decomposition) can entail a substantial computational overhead.

Intuitively, $\|c\|_1$ measures how far the non-Clifford gate is from the Clifford set. The farther it is, the more difficult the simulation becomes.

Combining the definition of stabilizer extension with Eq.18, we obtain:

$$\xi(U|0^n) \leq \xi(U) \leq \prod_j \xi(V_j). \quad (19)$$

Thus, decomposing each non-Clifford gate individually yields an upper bound on the circuit's overall optimal decomposition.

5.2 Sum Over Cliffords

The problem we wish to solve is, given a non-Clifford gate, ρ , and $\{A_i\}_i$ a Clifford group, find

$$\min \left\{ z = \sum_i |x_i| \right\} \quad \text{subject to} \quad \rho = \sum_i x_i A_i. \quad (20)$$

Depending on convenience, we will address several cases: letting $x_i \in \mathbb{C}$ or $x_i \in \mathbb{R}$, and choosing $\{A_i\}$ to be the Clifford group, the diagonal Clifford group or the real diagonal Clifford group.

The value of the ℓ^1 -norm (or its square) has a direct impact on the complexity of simulating the quantum circuit using the sum-over-Clifford method. This motivates the interest in studying it. The objective is to find the optimal decomposition of the non-Clifford gates.

5.3 Linear Programming

Linear programming is defined as the optimization of a linear function of certain variables subject to linear constraints. Canonically, any linear problem can be put into the following form:

$$\begin{aligned} & \text{Maximize} && z = c^T x, \\ & \text{subject to} && Ax = b, \\ & && x \geq 0, \end{aligned} \tag{21}$$

where A is the matrix of coefficients (or constraint matrix), x , c , and b are column vectors, the components of x are called *decision variables*, and z is the *objective function*.

Referring Eq.20, the optimization problem is in general a non-linear convex problem due to non-linear objective function and mainly the complex coefficients. In this project, the most general optimization is solved for one and two qubit gates. Specifically, we obtain the decomposition of the P gate, $P \otimes P$, and the $CP(\theta)$ gates using the CVX library [17]. However, as explained in section 4.3, the number of Clifford gates for three or more qubits is too large to be feasible on a classical computer. To obtain the decomposition for 3 or more qubit gates, we opt to decompose real diagonal gates using the real diagonal Clifford group. Furthermore, in that scenario, we can reduce problem 20 to a linear program in canonical form. As stated, the objective function is z is not strictly linear because of the absolute values $|x_i|$. Therefore, we must convert the absolute values into a linear form. A convenient method, with negligible effect on computational time, is:

1. Each variable x_i (originally $x_i \in \mathbb{R}$) is replaced by $x_i = x_i^+ - x_i^-$ in the linear constraints $Ax = b$.
2. In the objective function, $|x_i|$ is replaced by $x_i^+ + x_i^-$. Thus, the new objective function becomes

$$z' = \sum_i (x_i^+ + x_i^-),$$

which is linear.

The only drawback is that the number of variables doubles.

In Problem 20, the matrix ρ (to be decomposed) is complex. In this project, we are focused on the decomposition of the multi-CZ gates whose diagonal elements are real. As such, we proceed without taking into account the complex part of the matrix.

Consider the decomposition of a real diagonal non-Clifford gate ρ , expanded in matrix form:

$$\begin{aligned} \begin{pmatrix} \rho_{1,1} & 0 & \cdots & 0 \\ 0 & \rho_{2,2} & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & \rho_{m,m} \end{pmatrix} &= x_1 \begin{pmatrix} a_{1,1} & 0 & \cdots & 0 \\ 0 & a_{2,2} & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & a_{m,m} \end{pmatrix}^{(1)} + x_2 \begin{pmatrix} a_{1,1} & 0 & \cdots & 0 \\ 0 & a_{2,2} & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & a_{m,m} \end{pmatrix}^{(2)} \\ &+ \cdots + x_n \begin{pmatrix} a_{1,1} & 0 & \cdots & 0 \\ 0 & a_{2,2} & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & a_{m,m} \end{pmatrix}^{(n)} \end{aligned} \tag{22}$$

where $\{(a_{i,j})^{(k)}\}$ are the real entries of the unitary matrices A_k . Note that all $a_{ij} = \rho_{ij} = 0$ if $i \neq j$.

Focusing on the diagonal elements, we obtain a set of linear equations, illustrated here schematically for each $\rho_{i,i}$:

$$\begin{aligned}
\rho_{1,1} &= x_1(a_{1,1})^{(1)} + x_2(a_{1,1})^{(2)} + \cdots + x_m(a_{1,1})^{(n)} \\
\rho_{2,2} &= x_1(a_{2,2})^{(1)} + x_2(a_{2,2})^{(2)} + \cdots + x_m(a_{2,2})^{(n)} \\
&\vdots \\
\rho_{m,m} &= x_1(a_{m,m})^{(1)} + x_2(a_{m,m})^{(2)} + \cdots + x_m(a_{m,m})^{(n)}
\end{aligned}$$

Next, we substitute $x_i = x_i^+ - x_i^-$. For instance, for each diagonal element (i, i) :

$$\rho_{i,i} = x_1^+(a_{i,i})^{(1)} - x_1^-(a_{i,i})^{(1)} + x_2^+(a_{i,i})^{(2)} - x_2^-(a_{i,i})^{(2)} + \cdots + x_m^+(a_{i,i})^{(m)} - x_m^-(a_{i,i})^{(m)} \quad (23)$$

As such, if there are n matrices, A_i , in the decomposition, and m diagonal elements, the final problem has $2n$ variables and m constraints. The vector c in the LP is a $2n$ -dimensional vector of ones.

While optimization with complex coefficients can, in principle, yield optimal decompositions of quantum gates, linear programming with real coefficients offers a far more practical approach. In many relevant cases, the coefficients of the optimal solution are already real, which means that the results obtained through linear programming coincide with those of the more computationally expensive convex problem. With this formulation, we simplify the original problem in Eq. 20 for real diagonal non-Clifford gates. The optimal values obtained using this approach coincide with those from solving the corresponding non-linear convex problem. In doing so, we exploit the symmetries of the problem to achieve this simplification.

5.4 Multi-CZ gates

The importance of CCZ is quite large, not only because it forms a universal set together with Clifford gates, but also because it is equivalent, under local Hadamard transformations, to the CCNOT (Toffoli) gate. Indeed, one has

$$\text{CCNOT} = (I \otimes I \otimes H) \text{CCZ} (I \otimes I \otimes H),$$

so any implementation of CCZ plus single-qubit Hadamards suffices to realize a Toffoli. The Toffoli gate is crucial in quantum information processing as it enables the implementation of complex arithmetic and error correction circuits.

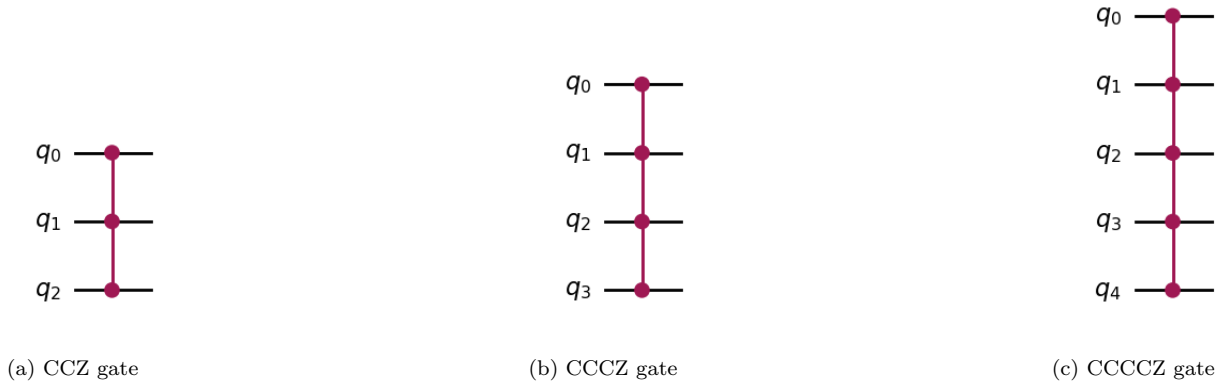


Figure 1. Multi CZ gates.

In addition, our formulation for decomposing real diagonal non-Clifford gates enables the decomposition of gates with up to six qubits without relying on HPC resources. Consequently, given a quantum circuit, we

can decompose large sections of the circuit that satisfy the criteria of being real and diagonal. This allows us to handle larger portions of the circuit compared to previous methods. In contrast, earlier approaches were limited to decomposing only one- or two-qubit gates, such as the T gate. Such methods are less efficient because they require a greater number of decompositions. The decomposed parts of the circuit can then be fed into a circuit simulator to simulate the full circuit. Moreover, decomposing larger sections of the circuit is computationally more efficient than first decomposing smaller sections and then multiplying them together. We provide supporting results in Section 6.3. Thus, the ability to decompose multi-CZ gates allows us to probe the limits of our method for decomposing real diagonal non-Clifford gates.

5.5 Solvers

In this project, the following four solvers were used to perform the decomposition of non-Clifford gates: HiGHS solver [18], GLPK solver [19], Clarabel solver [20], and CVX solver [17]. The HiGHS, GLPK and Clarabel solvers were used to decompose multi CZ-gates. In addition, the CVX package was used to perform the decomposition of the one and two qubits gates. The HiGHS solver was also used to obtain the decomposition of $CCZ \otimes CZ$ and $CCZ \otimes CCZ$ gates.

HiGHS is a software used for solving large scale sparse linear optimization models. HiGHS has efficient implementations of both the primal and dual simplex methods [18], although the dual simplex solver is likely to be faster and is more robust, so is used by default.

GLPK (GNU Linear Programming Kit) is a set of routines written in the ANSI C programming language and organized in the form of a callable library. It is intended for solving linear programming (LP), mixed integer programming (MIP), and other related problems.

CVX is a Matlab-based modeling system for convex optimization. CVX turns Matlab into a modeling language, allowing constraints and objectives to be specified using standard Matlab expression syntax.

And lastly, the CVXPY is a Python library used for solving convex optimization problems [21]. It can also be used to LP problems. It works by automatically transforming the problem into standard form, calls a solver, and unpacks the results. It is possible to specify different solvers to solve the optimization problem. In this project, the Clarabel solver was used.

5.6 Profiling

The CVXPY and HiGHS libraries were implemented in Python. To measure performance, we used Scalene [22], a low-overhead profiler that reports both execution time and memory usage. Scalene provides detailed profiling at the function and line level, supports multiprocessing tasks, and requires no code modifications. It can be run directly from the command line, making it convenient to use.

5.7 Hardware Specification

The scripts were containerized in a docker container. The Dockerfile contains all the required libraries needed to reproduce the results, including the libraries for the solvers and the profiler. The code for this project can be accessed at this GitHub repository [23].

All the results were obtained using a MacBook Pro with an Apple M1 Max chip, 32 GB memory, and 10 cpu cores.

6 Result

All results obtained in this section was obtained by the author. The corresponding source code can be found in the Github repository [23]. In this project, we have used MATLAB, Python and C. The implementation of the CVX solver was written in MATLAB. Python was used to implement the HiGHS solver and the CVXPY library; and the GLPK solver was implemented using C. To reproduce the results presented in this work, you first need to build the Docker image provided in the codebase. This image includes all the required libraries to run the scripts. Once built, the scripts can be executed from a Docker container created from this image.

Before discussing the results in detail, we briefly mention the gates and circuits under study in this project.

The non-Clifford gates we will investigate are the following: $P(\theta)$, $CP(\theta)$, $\sqrt{\text{SWAP}}$, and CCZ . The parameter θ is the relative phase introduced by the gates, $P(\theta)$ and $CP(\theta)$, between the states $|0\rangle$ and $|1\rangle$. The representation of the gates in the computational basis are:

$$P(\theta) = \begin{pmatrix} 1 & 0 \\ 0 & e^{i\theta} \end{pmatrix}, \quad (5.1)$$

$$CP(\theta) = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & e^{i\theta} \end{pmatrix}, \quad (5.2)$$

$$\sqrt{\text{SWAP}} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & \frac{1}{2}(1+i) & \frac{1}{2}(1-i) & 0 \\ 0 & \frac{1}{2}(1-i) & \frac{1}{2}(1+i) & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}. \quad (5.3)$$

Highlighting some properties:

$$P\left(\frac{\pi}{4}\right)^n = T^n, \quad P\left(\frac{\pi}{2}\right)^n = S^n, \quad CP(\pi) = CZ, \quad CP(0) = CP(2\pi) = I. \quad (5.4)$$

For multi-controlled-Z gates (diagonal only):

$$CCZ = \text{diag}(1, 1, 1, 1, 1, 1, 1, -1),$$

$$CCCZ = \text{diag}(\underbrace{1, 1, \dots, 1}_{15 \text{ ones}}, -1),$$

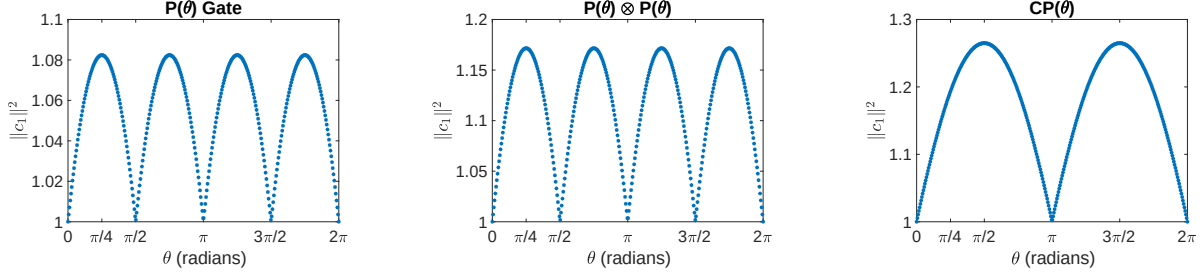
$$CCCCZ = \text{diag}(\underbrace{1, 1, \dots, 1}_{31 \text{ ones}}, -1),$$

$$CCCCCZ = \text{diag}(\underbrace{1, 1, \dots, 1}_{63 \text{ ones}}, -1).$$

6.1 Single and Two-Qubit Gates

To begin, we solve Problem 20 using complex coefficients for one- and two-qubit non-Clifford gates. The objective here is to verify that the decomposition that we obtain agrees with the literature. For three or more qubits, the Clifford group cardinality becomes too large to store all the matrices that might appear in the decomposition.

We present the optimal value ξ for the following gates: $P(\theta)$, $P(\theta) \otimes P(\theta)$ and $CP(\theta)$ gates. In addition to computing the optimal decomposition, one needs the explicit decomposition itself to apply the Sum-Over-Cliffords method. In theory, there are known results for the T gate, which is particularly relevant for forming a universal set with Clifford gates.



(a) $P(\theta)$ gate

(b) $P(\theta) \otimes P(\theta)$ gate

(c) $CP(\theta)$ gate

Figure 2. The optimal values of ξ . (a) $P(\theta)$ gate (b) $P(\theta) \otimes P(\theta)$ gate (c) $CP(\theta)$ gate.

Figure 2 shows how the stabilizer extent changes with the introduced phase θ . For $P(\theta)$, the maximum value of the stabilizer extension appears at $\theta = \pi/4$, i.e. when it becomes the T gate. We obtain the optimal decomposition for the T gate.

$$\begin{pmatrix} 1 & 0 \\ 0 & e^{i\pi/4} \end{pmatrix} = c \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} + c^* \begin{pmatrix} 1 & 0 \\ 0 & -i \end{pmatrix}, \quad (24)$$

where $c \approx 0.5000 - 0.2071i$. The optimal values for the $P(\theta) \otimes P(\theta)$ also gives similar trend as the optimal values for the $P(\theta)$ gate. The maximum also occurs at $\pi/4$. What is worth noting is that the 1-norm obeys the multiplicativity as given in Eq.18. For instance, the 1-norm for $P(\pi/4)$ is 1.083 and the 1-norm for $P(\pi/4) \otimes P(\pi/4)$ is 1.172.

In other words, the stabilizer extension exhibits *multiplicativity* under the tensor product for this gate. The strictly submultiplicative property in Eq.19 states

$$\xi(U_1 \otimes U_2) \leq \xi(U_1) \cdot \xi(U_2),$$

but we have found (numerically) that it also appears to satisfy

$$\xi(U_1 \otimes U_2) = \xi(U_1) \times \xi(U_2),$$

at least as presented in Fig.2. The decomposition of the $CP(\theta)$ gate gives maximum values at $\pi/2$ and $3\pi/2$. At $\theta = \pi$, we have the CZ gate, which is a Clifford gate. Hence, the optimal value at this point is 1.

A recurring question arises: which gate is ideal for conferring universality upon a Clifford circuit? The discussion is nontrivial because it depends on what gates one wishes to implement. In [16], it is shown that using T gates to implement CCZ is more costly than implementing CCZ directly. This is logical, given the submultiplicative property for products of gates. Thus, in principle, the best approach is to implement the desired gate directly. But in reality, this might be impossible or even impractical, and it is often advantageous to minimize the number of distinct gates one must physically implement. Table 6.4 summarizes our numerical results for some important non-Clifford gates:

Gate	ξ
T	1.172
CCZ	1.778
$CP(\frac{\pi}{2})$	1.600
$\sqrt{\text{SWAP}}$	1.600

6.2 Decomposition of multi-CZ gates

Here, we present the decomposition of multi-CZ gates up to 6 qubits. As shown in Table 2, the diagonal Clifford group for 6 qubits contains more than 100 million elements. Since multi-CZ gates are both real and diagonal, we can further restrict our attention to the *real diagonal Clifford group*, which drastically reduces the number of elements. For six qubits, this reduction yields 2,097,152 elements, each of which can be represented by 64 integer entries corresponding to its diagonal. This compression makes it feasible to perform decompositions for larger qubit systems.

For the case of multi-CZ gates, the diagonals are restricted to values of $+1$ or -1 . However, extending this method to 7 qubits would still result in prohibitive memory requirements, making such decompositions intractable on standard computational resources. Table 3 gives the number of elements in the real diagonal Clifford group up to six qubits.

Table 3. Number of elements in the real diagonal Clifford group for n qubits.

Number of qubits (n)	Number of elements
2	32
3	64
4	1024
5	32,768
6	2,097,152

The following solvers are used to obtain the decomposition: HiGHS solver, GLPK solver and the Clarabel solver. We track the optimal value of $\xi = \|c\|_1^2$, the time to obtain the solution and the maximum memory required by each solver to obtain the decomposition. For the memory usage, we only present results for the HiGHS and Clarabel solvers.

Table 4. Decomposition time and memory usage of all three solvers for the multi-CZ gates.

Gate	Time CBL (s)	Time HiGHS (s)	Time GLPK (s)	Mem CBL (MB)	Mem HiGHS (MB)
CCZ	0.0537	0.0025	0.000932	0*	0*
CCCZ	0.413	0.0455	0.00562	7	10
CCCCZ	4.95	0.970	0.357	505	613
CCCCCZ	7760	351	145	49000	75000

Figure 3 shows that the optimal decomposition norm, $\|c\|_1^2$, increases steadily with the number of qubits, ranging from 1.778 for the CCZ gate to 2.641 for the CCCCCZ gate. This confirms the expected growth in complexity as the gates become larger. Also, all three solvers give similar decomposition values. As such, one would expect that the number of Clifford gates used in the decomposition for all three solvers to be similar. Table 5 shows the number of Clifford gates used in the decomposition of the multi-CZ gates for each solver. From the results, it is evident that although all three solvers—HiGHS, GLPK, and Clarabel—achieve identical norm values for the decomposition of multi-CZ gates, the number of non-zero elements in the resulting decompositions varies significantly. For instance, in the case of the CCCCCZ gate, each solver reaches the same optimal norm of 2.641, yet HiGHS and GLPK obtain relatively sparse solutions with 62 and 63 non-zero elements, respectively, while Clarabel produces a much denser decomposition with 652 elements. This indicates that while the optimization landscape allows for multiple valid solutions yielding the

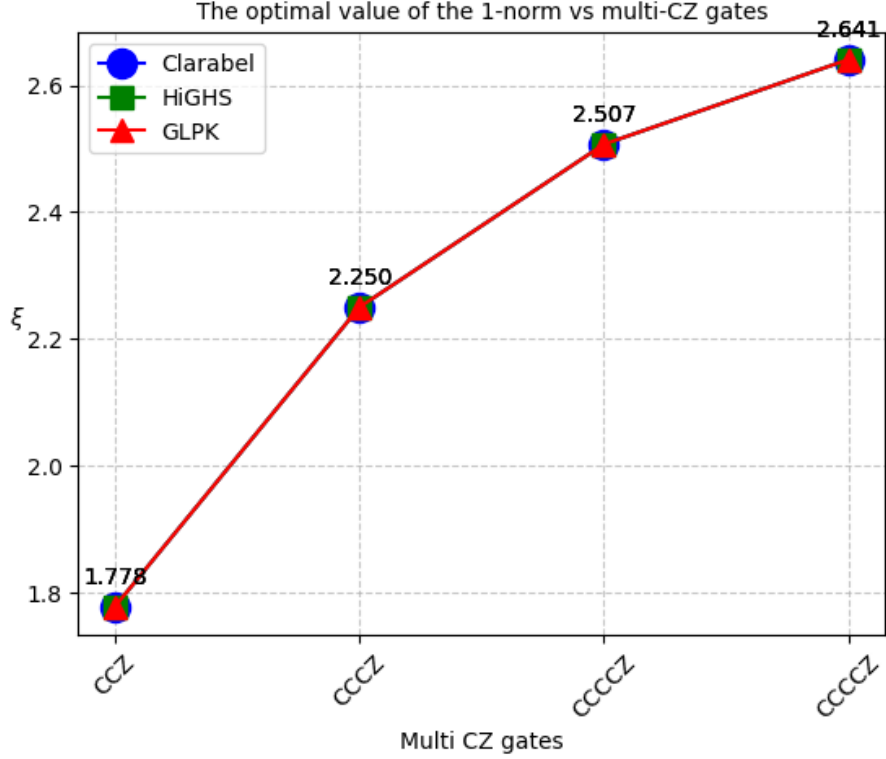


Figure 3. This figure shows the optimal decomposition value, ξ , of the multi CZ gates

same objective value, the sparsity of the decomposition depends strongly on the solver. HiGHS and GLPK are thus more effective at producing compact decompositions, which is advantageous for simulation efficiency, whereas Clarabel tends to generate solutions with larger support, leading to greater resource requirements.

	CCZ	CCCZ	CCCCZ	CCCCCZ
HIGHS	8	6	28	62
GLPK	8	16	32	63
Clarabel	8	36	156	652

Table 5. Number of non-zero elements in the decomposition.

Table 4 highlights the disparity in solver performance in terms of computation time and memory usage. The GLPK solver required the minimum amount of time to obtain the optimal solution for all the decompositions. It only required 145 seconds to obtain the decomposition of the CCCCCZ gate compared to the 351 seconds used by the HiGHS solver and the 7763 seconds used by the Clarabel solver. For the Clarabel solver, it should be noted that we accessed it through the third-party software CVXPY, which may have introduced additional overhead in execution time. For future benchmarks, the use of third-party interfaces to access solvers should be avoided whenever possible, as they are likely to cause similar time overheads, as observed here. For the memory usage, we only present results for the Clarabel and HiGHS solvers. The memory usage in general was higher for the HiGHS solver compared to the Clarabel solver. Also, the memory usage for decomposing the CCZ gates was insignificant can highlighted in Table 4 as 0*. These results demonstrate that while all solvers achieve consistent norm values, their efficiency in terms of time, memory consumption, and sparsity of decomposition varies considerably, with HiGHS and GLPK outperforming Clarabel in scalability. For HiGHS in particular, this highlights the fact that it is a solver designed for larger problems.

6.3 Multiplicity and Decomposition of Circuits

The multiplicity of norms arising from the tensor product of gates provides an important consistency check for the sum-over-Cliffords framework. As shown in Table 6, the decomposition of $\text{CCZ} \otimes \text{CCZ}$ yields a norm of 3.160, while $\text{CCZ} \otimes \text{CZ}$ gives 1.778. These values are fully consistent with the apparent multiplicity rule for tensor products, which states that the stabilizer extension satisfies

$$\xi(U \otimes V) = \xi(U) \cdot \xi(V). \quad (25)$$

For instance, since $\xi(\text{CCZ}) = 1.778$ and $\xi(\text{CZ}) = 1.000$ (Clifford gates decompose exactly with unit norm), the product $\xi(\text{CCZ}) \cdot \xi(\text{CZ}) = 1.778$, matching precisely the observed value for $\text{CCZ} \otimes \text{CZ}$. Similarly,

$$\xi(\text{CCZ})^2 = (1.778)^2 = 3.160,$$

which is in excellent agreement with the computed norm for $\text{CCZ} \otimes \text{CCZ}$. This confirms that the multiplicity property is preserved for these examples, validating both the correctness of the decomposition and the reliability of the solver framework.

In addition to these tensor-product decompositions, I also obtained results for the decomposition of entire circuits consisting of multiple layers of CZ-type gates as shown in Figure 4. The results are:

$$\text{Figure4}(a) = 3.160,$$

$$\text{Figure4}(b) = 2.560,$$

$$\text{Figure4}(c) = 1.778,$$

A key observation is that the optimal value of the decomposition of the entire circuit is consistently smaller than the product of the optimal values of the individual CCZ layers. For example, a six-qubit circuit composed of four CCZ gates would naively yield a multiplicity of $(1.778)^4 = 9.989$, but the full decomposition of the circuit in Fig. 4(a) is 3.160. Similarly, Fig. 4(b) produces a norm of 2.560, which is significantly smaller than the product of norms obtained if each gate were decomposed independently. This demonstrates that global decomposition of entire circuits can exploit structural symmetries and redundancies between layers, leading to more compact stabilizer representations. Such results highlight the practical advantage of treating whole circuits rather than individual gates when aiming to minimize simulation complexity.

This highlights the importance of our approach: decomposing larger sections of a circuit at once yields a more efficient overall decomposition than decomposing smaller gates, such as the T gate, individually and then combining them afterward. The only restriction on these sections is that they must be real and diagonal, which ensures that the decomposition remains computationally feasible.

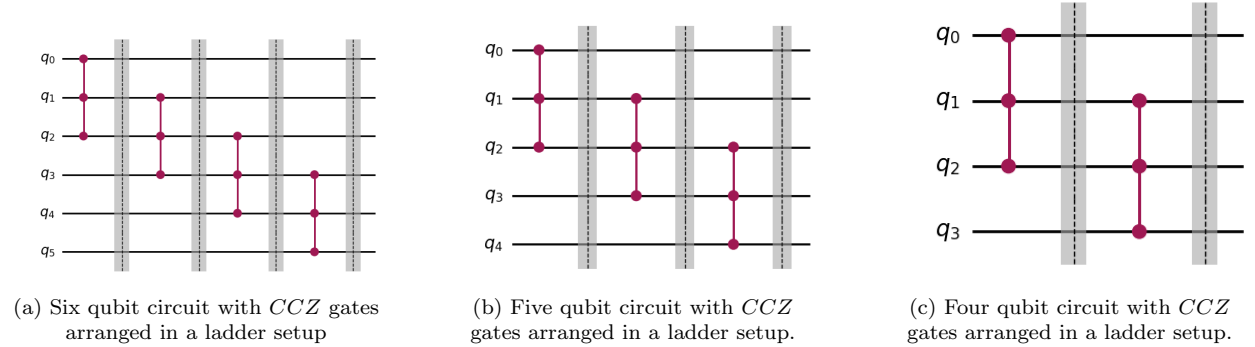


Figure 4

Tensor product of multi CZ gates	ξ
$ccz \otimes ccz$	3.160
$ccz \otimes cz$	1.778

Table 6. Decomposition of tensor product of multi-CZ gates.

6.4 Parallelisation of the HIGHS solver

HiGHS provides the option to enable parallelization by specifying the number of threads to be used during the optimization process. This feature allows the solver to exploit multiple cores, significantly reducing solution times for large problems. Next, I tracked the time to solution and the memory usage of the HIGHS solver as the number of threads increases; using the scalene profiler. This was done specifically for the CCCCCZ gate, as it contains the largest number of Clifford gates in its decomposition. The objective here is to assess the scalability of the HIGHS solver and evaluate how best an HPC cluster could be employed for such computations. The profiling provides insight into the solver’s computational demands as the problem size increases, offering a practical benchmark for distributing workloads and allocating resources efficiently on HPC systems.

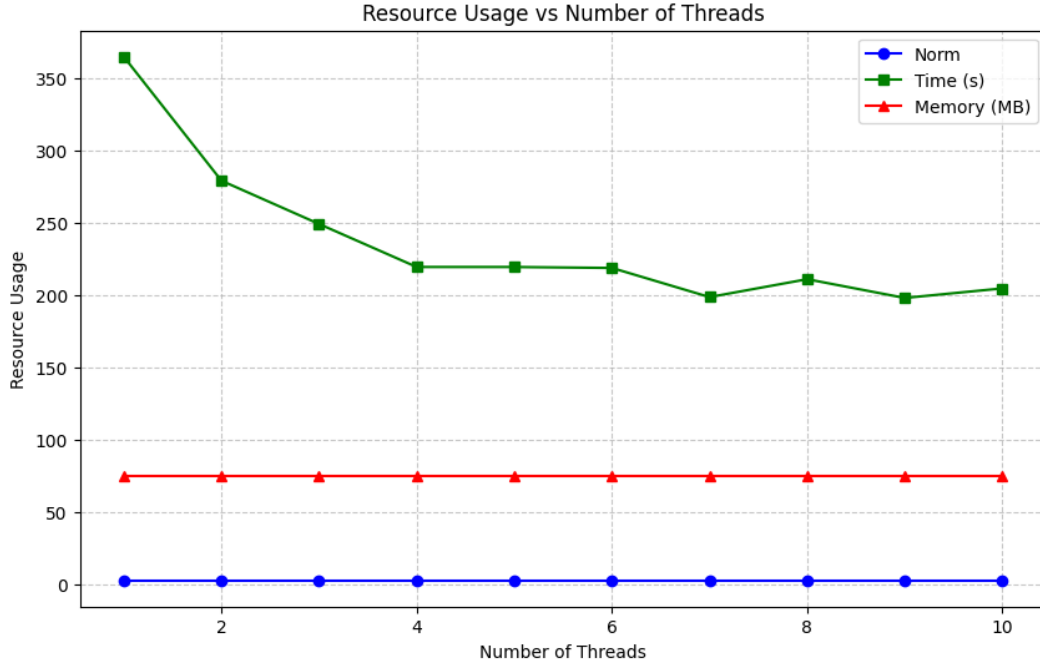


Figure 5. Parallelization of the HIGHS solver: we track the optimal value, solution time, and memory usage as the number of threads increases from 1 to 10.

The results in Figure 5 illustrate the effect of thread parallelization on the performance of the HiGHS solver for the decomposition of the CCCCCZ gate. As shown, the norm of the decomposition remains constant across all thread counts, confirming that parallelization does not affect the quality of the solution. In contrast, the time-to-solution decreases significantly as the number of threads increases, with the largest improvement observed between one and four threads, after which the runtime stabilizes around 200–220 seconds. This demonstrates the benefit of parallelization in reducing computational time, though with diminishing returns beyond a certain number of threads. Memory usage remains largely constant at approximately 75 MB regardless of thread count, indicating that the parallelization strategy of HiGHS primarily impacts execution speed rather than memory requirements. These findings highlight that while HiGHS can effectively exploit multiple cores to accelerate decomposition tasks, careful tuning of the number of threads is necessary to balance performance gains with efficient use of HPC resources.

7 Discussion & Conclusion

The results presented in this thesis demonstrate that the *sum-over-Clifford* approach provides a systematic and scalable framework for simulating quantum circuits dominated by Clifford operations but augmented by a limited number of non-Clifford gates. By reformulating the decomposition of non-Clifford unitaries into an optimization problem over Clifford operators, we obtained explicit stabilizer decompositions of single- and two-qubit gates as well as multi-qubit controlled-Z (CZ) gates.

A key finding is that the *stabilizer extension* ($\|c\|_1^2$ norm) grows moderately with the number of qubits for multi-CZ gates, from 1.778 for the CCZ gate to 2.641 for the five-controlled-Z gate. This confirms the expected increase in classical simulation cost with circuit complexity, yet the growth remains sub-exponential for the gates studied. Moreover, tensor-product decompositions were shown to obey the multiplicative property, as expected, and confirming the robustness of the decomposition procedure; whether the multiplicativity is strict or not in general is an open problem in the literature. Interestingly, global decompositions of full circuits often produced significantly smaller stabilizer extensions than the product of decomposed gates, highlighting the importance of exploiting *circuit-level symmetries* rather than treating gates in isolation.

The choice of solver was found to play a crucial role in determining both efficiency and sparsity of the decompositions. While HiGHS and GLPK produced compact decompositions with few non-zero terms, Clarabel tended to generate much denser solutions, despite achieving identical optimal norms. This underscores the necessity of solver selection in practical implementations: sparse decompositions directly translate to more efficient stabilizer simulations. Additionally, profiling experiments confirmed that parallelization substantially improves solver performance, though with diminishing returns beyond four threads. These insights suggest that scaling the method to larger systems will benefit most from *high-performance computing environments* with carefully tuned solver configurations.

From a broader perspective, these findings contribute to the ongoing effort to map the boundary between classically simulable and intractable quantum circuits. The sum-over-Clifford method bridges the gap between efficiently simulable stabilizer circuits and fully universal circuits, providing a quantitative tool to assess the “distance” of a gate or circuit from the Clifford group. In this regard, the stabilizer extension $\xi(W)$ serves as a resource measure for non-Cliffordness, aligning with recent developments in resource theories of quantum computation[16, 24].

Nevertheless, several limitations remain. The exponential growth of the Clifford group restricts exact decomposition to relatively small systems, even when diagonal subgroups are used. While the exploitation of circuit symmetries mitigates this growth, further algorithmic advances are required to scale beyond the six-qubit gates explored here. Moreover, the focus on (real) diagonal gates, while practical, leaves open the extension of these techniques to more general classes of non-Clifford unitaries.

In conclusion, this thesis demonstrates that the sum-over-Clifford method, when combined with symmetry exploitation and modern optimization techniques, enables efficient classical simulation of moderately sized non-Clifford circuits. The approach is both conceptually illuminating and practically useful: it clarifies the role of non-Clifford resources in quantum computation while offering concrete strategies for simulating near-term quantum devices. Future work may extend these results in three main directions:

1. **Generalization:** extend the decomposition framework to broader gate families beyond diagonal operators;
2. **Integration:** combine with tensor network methods or Monte Carlo sampling to reduce memory overhead;
3. **Deployment:** leverage HPC clusters to push the frontier of classically simulable circuits to larger system sizes.

Ultimately, the insights gained here reinforce the dual role of Clifford-dominated circuits: as a testbed for algorithm design and as a boundary marker for quantum advantage. By quantifying the cost of non-Clifford resources, the sum-over-Clifford framework deepens our understanding of the interplay between classical and quantum computational power.

References

- [1] Lutz Bornmann et al. “Quantum technology—a bibliometric analysis of a maturing research field”. In: *Max Planck Society* (2019).
- [2] Vikas Hassija et al. “Present landscape of quantum computing”. In: *IET Quantum Communication* 1.2 (2020), pp. 42–48.
- [3] Nathalie P De Leon et al. “Materials challenges and opportunities for quantum computing hardware”. In: *Science* 372.6539 (2021), eabb2823.
- [4] “Quantum error correction below the surface code threshold”. In: *Nature* 638.8052 (2025), pp. 920–926.
- [5] Ethan Lee and Jothi Ramaswamy. “Classical Simulation of Quantum Stabilizer and Clifford Circuits”. In: ().
- [6] Scott Aaronson. “BQP and the polynomial hierarchy”. In: *Proceedings of the forty-second ACM symposium on Theory of computing*. 2010, pp. 141–150.
- [7] Jianxin Chen et al. “Classical simulation of intermediate-size quantum circuits”. In: *arXiv preprint arXiv:1805.01450* (2018).
- [8] Daniel Gottesman. “Theory of fault-tolerant quantum computation”. In: *Physical Review A* 57.1 (1998), p. 127.
- [9] Sergey Bravyi and Alexei Kitaev. “Universal quantum computation with ideal Clifford gates and noisy ancillas”. In: *Physical Review A—Atomic, Molecular, and Optical Physics* 71.2 (2005), p. 022316.
- [10] Scott Aaronson and Daniel Gottesman. “Improved simulation of stabilizer circuits”. In: *Physical Review A—Atomic, Molecular, and Optical Physics* 70.5 (2004), p. 052328.
- [11] Artur Ekert and Richard Jozsa. “Quantum computation and Shor’s factoring algorithm”. In: *Reviews of Modern Physics* 68.3 (1996), p. 733.
- [12] Iulia M Georgescu, Sahel Ashhab, and Franco Nori. “Quantum simulation”. In: *Reviews of Modern Physics* 86.1 (2014), pp. 153–185.
- [13] Camillo et al. *Work in preparation*. Personal communication, email: jbermejovega@go.ugr.es. 2025.
- [14] Christopher Granad. *python-quaec*. 2025. URL: <https://github.com/cgranade/python-quaec> (visited on 09/06/2025).
- [15] Michael A Nielsen and Isaac L Chuang. *Quantum computation and quantum information*. Cambridge university press, 2010.
- [16] Sergey Bravyi et al. “Simulation of quantum circuits by low-rank stabilizer decompositions”. In: *Quantum* 3 (2019), p. 181.
- [17] CVX. *CVX: Matlab Software for Disciplined Convex Programming*. 2020. URL: <https://cvxr.com/cvx/> (visited on 09/06/2025).
- [18] Qi Huangfu and JA Julian Hall. “Parallelizing the dual revised simplex method”. In: *Mathematical Programming Computation* 10.1 (2018), pp. 119–142.
- [19] GLPK. *GLPK (GNU Linear Programming Kit)*. 2025. URL: <https://www.gnu.org/software/glpk/> (visited on 09/06/2025).
- [20] clarabel. *Clarabel Solver*. 2025. URL: <https://clarabel.org/stable/> (visited on 09/06/2025).
- [21] cvxpy. *CVXPY*. 2025. URL: <https://www.cvxpy.org/index.html> (visited on 09/06/2025).
- [22] Emery D. Berger, Sam Stern, and Juan Altmayer Pizzorno. “Triangulating Python Performance Issues with Scalene”. In: *17th USENIX Symposium on Operating Systems Design and Implementation (OSDI 23)*. Boston, MA: USENIX Association, July 2023, pp. 51–64. ISBN: 978-1-939133-34-2. URL: <https://www.usenix.org/conference/osdi23/presentation/berger>.
- [23] Boaz Micah. *Symmetry-optimized classical simulation of quantum circuits*. 2025. URL: <https://github.com/Boaz-ato/Sum-over-clifford-groups> (visited on 09/06/2025).
- [24] Michael Beverland et al. “Lower bounds on the non-Clifford resources for quantum computations”. In: *Quantum Science and Technology* 5.3 (2020), p. 035009.